

Architectural Justification: Incident Escalation & On-Call Rotation Engine

Lab 3: Component Modelling & Architectural Pattern Selection · Problem Statement #48

Guru R Bharadwaj · PES1UG24CS177 · Section 5C

Architecture Selection: We chose the **Microservices architecture** for the Incident Escalation & On-Call Rotation Engine.

Style analysis

Style	Assessment against this scenario
Layered	Clear separation and simple to build, but every request shares one business layer and one database. A slow post-mortem report query would contend with alert routing, and the whole application has to be scaled and redeployed as a unit. That does not suit a paging path that has to survive its own release cycle.
Client-server	Centralised control and easy consistency, but the single server is a single point of failure. A tool that pages engineers when production breaks cannot itself be less available than the services it watches (NFR-002 asks for 99.9%).
Microservices	Selected. The system splits along boundaries that are already stable: ingestion, escalation, scheduling, notification and storage. Each one has a different load profile and a different tolerance for failure. The extra operational complexity buys the availability the requirement asks for.

Two reasons for the choice

- Fault isolation on the paging path.** The escalation path (Alert Ingestion → Escalation Engine → Notification Gateway) is the flow that has to keep working, because an unrouted P1 alert is the failure mode the system exists to prevent (FR-001). Isolating it as its own set of services means a defect or an outage in the low-criticality Web Console & Post-Mortem Service cannot exhaust the escalation engine's resources or block its deployment. Under a layered or client-server design the two share one process and one failure domain.
- Divergent load and change profiles.** Alert ingestion and notification dispatch are bursty and I/O bound, running to 200 alerts per minute (NFR-001), with each alert fanning out to a slow external carrier. The Rotation Scheduler and the post-mortem workflows are low-volume CRUD used by a handful of Incident Commanders. Independent deployability lets us run many replicas of the ingestion and gateway services during an alert storm and a single replica of the rest, and lets the escalation ladder logic (FR-004) ship without redeploying the scheduler.

Security advantage

Microservices confine the sensitive assets to one component and narrow the blast radius of a compromise. Engineers' personal phone numbers and the carrier API credentials live only inside the **Notification Gateway Service**, which is reachable through the internal *INotificationDispatch* interface and has no route from the public internet. The internet-facing **Web Console** never holds those secrets, so compromising the browser-facing surface does not yield credentials or responder PII. The same boundary serves NFR-002's audit requirement: the **Incident & Audit Store** exposes only *IIncidentRepository*, so every schedule change, notification attempt and acknowledgement passes through a single append-only interface that can be mutually authenticated with mTLS and logged, instead of through direct database access spread across a shared business layer.

Performance benefit

Decoupling ingestion from dispatch makes the three-second dispatch target (NFR-001) achievable. *INotificationDispatch* runs asynchronously over a queue, so the Alert Ingestion Service acknowledges the monitoring webhook and hands the incident off immediately, without blocking on carrier round-trips that can run to several seconds. The Notification Gateway then fans out to SMS, voice and e-mail in parallel. The ingestion and gateway services also scale horizontally on their own, so a burst of 200 alerts per minute is absorbed by adding replicas to those two services alone. A layered deployment would have to scale the whole application, including the idle post-mortem and scheduling code, to add the same dispatch throughput.

Trade-off: the design accepts higher operational complexity and eventual consistency between services. Both are contained by keeping incident state authoritative in one Incident & Audit Store instead of partitioning it per service.